PROJET D'IHM 2026 · DOSSIER DE CONCEPTION TECHNIQUE

Architecture des fichiers

Organisation du code de la plateforme d'apprentissage (Java · Python · C++)
application de bureau JavaFX 21 · thèmes AtlantaFX · style CSS3

1 vue = 1 FXML + 1 contrôleur | MVVM | style externalisé | client non digne de confiance | 8 écrans · 3 rôles

1 · L'arborescence complète

Une seule racine Maven, quatre zones : `src/main/java` (le code), `src/main/resources` (le style et les textes), `src/test/java` (les tests) et `docs/` (les livrables de conception, versionnés avec le code). Les lignes grises sont des commentaires : elles disent **à quoi sert** chaque fichier, pas seulement son nom.

1.1 Le tronc : racine, documentation, socle technique

```
ihm-2026/                          racine du dépôt (Maven)
├── pom.xml                        JavaFX 21 · AtlantaFX · Jackson · TestFX · jpackage
├── README.md                      lancement, thèmes, conventions du projet
├── docs/                          livrables de conception, versionnés avec le code
│   ├── analyse-besoins-IHM.pdf · canvas-document-analyse-besoins.pdf
│   ├── Diagrammes-du-projet.pdf            7 diagrammes UML
│   └── Modeles-de-taches-du-projet-IHM.pdf 8 modèles HTA
├── src/main/java/fr/ihm2026/
│   ├── module-info.java             requires javafx.controls, javafx.fxml, atlantafx.base…
│   ├── App.java                     point d'entrée : Scene, thème, router, session
│   ├── core/                        transverse — ne dépend d'aucun autre paquet
│   │   ├── Router.java                    navigation, historique, fil d'Ariane
│   │   ├── Session.java                   utilisateur, rôle, parcours courant, jeton
│   │   ├── ThemeManager.java              thème AtlantaFX clair/sombre + préférence
│   │   ├── TaskRunner.java                hors FX thread + Platform.runLater
│   │   └── i18n/UiText.java               accès aux libellés français
```

1.2 Le cœur applicatif : données, modèles, interface

```
src/main/java/fr/ihm2026/  (suite)
│   ├── api/                        client HTTP/JSON — seul contact avec le serveur
│   │   ├── ApiClient.java                 GET/POST, jetons, erreurs normalisées
│   │   ├── ApiException.java
│   │   ├── dto/  UtilisateurDto · ParcoursDto · NiveauDto · CoursDto
│   │   │         QuestionDto · TentativeDto · ResultatDto · JournalDto
│   │   └── service/ AuthApi · CoursApi · TestApi · ProgressionApi · AdminApi
│   │       └── mock/                réponses simulées : maquetter sans serveur
│   ├── model/                      modèles observables (propriétés JavaFX)
│   │   ├── NiveauVM · CoursVM · QuestionVM · UtilisateurVM
│   │   ├── TentativeVM · ResultatVM
│   │   └── EtatNiveau.java                VERROUILLE · EN_COURS · VALIDE
│   └── ui/                         1 vue = 1 FXML + 1 contrôleur + 1 bloc CSS
│       ├── commun/                  composants partagés par les trois rôles
│       │     NavBar · BadgeLangage · CarteNiveau · EtatVide
│       │     DialogueConfirmation · SelecteurParcours  (IHM-21)
│       ├── apprenant/
│       │   ├── inscription/  InscriptionView.fxml + controller
│       │   ├── parcours/     ParcoursView.fxml + FriseNiveaux.java
│       │   ├── cours/        CoursView.fxml + SommaireTree.java
│       │   ├── exercice/     ExerciceView.fxml + EditeurCode + ConsoleSortie
│       │   ├── test/         TestView.fxml + TestController + Minuteur
│       │   ├── resultat/     ResultatView.fxml + controller
│       │   └── tableaubord/  TableauBordApprenantView.fxml + controller
│       ├── enseignant/
│       │   ├── editeurcours/ EditeurCoursView.fxml + controller + Autosave
│       │   ├── editeurtest/  EditeurTestView.fxml + QuestionEditor
│       │   ├── conditions/   ConditionsView.fxml  seuil · tentatives · période
│       │   └── resultats/    ResultatsView.fxml   KPI + TableView
│       ├── admin/   ComptesView · JournalAuditView · AgentIaView (.fxml + controllers)
│       └── ia/      PanneauAssistant.fxml + PanneauAssistantController
```

1.2.1 Les dépendances déclarées

```
module fr.ihm2026 {
    requires javafx.controls;
    requires javafx.fxml;
    requires atlantafx.base;
    requires com.fasterxml.jackson.databind;

    opens fr.ihm2026.ui to javafx.fxml;   // FXML instancie les contrôleurs
    opens fr.ihm2026.api.dto to com.fasterxml.jackson.databind;
    exports fr.ihm2026;
}
```

Le module JavaFX déclare ses besoins et n'ouvre que ce que FXML et Jackson doivent voir.

```
<dependencies>
  <dependency>
    <groupId>org.openjfx</groupId><artifactId>javafx-controls</artifactId>
    <version>21.0.4</version>
  </dependency>
  <dependency>                          <!-- thèmes clair / sombre -->
    <groupId>io.github.mkpaz</groupId><artifactId>atlantafx-base</artifactId>
    <version>2.0.1</version>
  </dependency>
  <dependency>                          <!-- client HTTP/JSON -->
    <groupId>com.fasterxml.jackson.core</groupId><artifactId>jackson-databind</artifactId>
    <version>2.17.2</version>
  </dependency>
  <dependency>                          <!-- tests d'interface -->
    <groupId>org.testfx</groupId><artifactId>testfx-junit5</artifactId>
    <version>4.0.18</version><scope>test</scope>
  </dependency>
</dependencies>
```

Quatre dépendances seulement : JavaFX, AtlantaFX, Jackson, TestFX.

Ces quatre lignes de `pom.xml` ont une conséquence directe sur le rangement : **JavaFX** fournit tous les contrôles (IHM-JFX1), **AtlantaFX** les thèmes (IHM-JFX2), **Jackson** impose l'existence du paquet `api/dto` (un DTO par objet reçu), **TestFX** impose que chaque écran soit atteignable sans clic préalable — d'où un `Router` qui prend un écran par son nom.

Dépendance	Ce qu'elle apporte	Ce qu'elle impose dans l'arborescence
javafx-controls / javafx-fxml	fenêtres, contrôles, chargement des FXML	un paquet `ui/` découpé en dossiers par rôle, un FXML par écran
atlantafx-base	thèmes clair et sombre, styles de contrôles	une couche `ThemeManager` + 2 feuilles de surcharge dans `css/`
jackson-databind	lecture/écriture JSON	un paquet `api/dto` de `record` immuables, isolé de la vue
testfx-junit5	tests d'interface (clics, saisies, assertions)	un paquet `src/test/…/ui` + des captures de référence

1.3 Les ressources, les tests et les outils

```
src/main/resources/fr/ihm2026/
├── css/
│   ├── app.css                     jetons de design + composants maison
│   ├── theme-light.css             nos surcharges du thème AtlantaFX clair
│   ├── theme-dark.css              nos surcharges du thème AtlantaFX sombre
│   └── syntax/ syntax-java.css · syntax-python.css · syntax-cpp.css
├── i18n/  messages_fr.properties
└── img/   logo.svg · icones/ · etats/ (valide · en-cours · verrouille)

src/test/java/fr/ihm2026/
├── ui/  InscriptionUiTest · TestValidationUiTest · ThemeUiTest
├── api/ ApiClientTest
└── resources/screenshots/       captures de référence (clair + sombre)

tools/
├── package-linux.sh · package-windows.ps1 · package-macos.sh
└── ci.yml                        compilation + TestFX headless
```

2 · Les quatre règles qui gouvernent ce rangement

Règle	Ce qu'elle impose	Pourquoi (exigence)
1 · Une vue, trois fichiers	Tout écran = `XxxView.fxml` + `XxxController.java` + un bloc CSS `.xxx`. Aucune logique dans le FXML, aucune construction d'interface en Java.	BNF-T9 (maintenabilité FXML/CSS), IHM-JFX1 (contrôles standards)
2 · Le sens des dépendances	`core` ← `api` ← `model` ← `ui` : une couche ne connaît que la précédente. `core` ne dépend **de personne**, `ui` n'appelle jamais la base.	BNF-09 (règles hors des écrans), BNF-T6 (client non digne de confiance)
3 · Zéro style dans le Java	Aucune couleur, taille ou marge en dur : tout est une classe CSS ou un jeton de `app.css`. Exception dynamique = commentée.	BNF-T4 (style externalisé), IHM-JFX3 (jetons de design)
4 · Les ressources ne sont pas du code	Textes en `messages_fr.properties`, images dans `img/`, feuilles de style dans `css/`. Un écran n'embarque aucun libellé en dur.	BNF-11 (langue), IHM-11 (cohérence des libellés)

2.1 Le rôle de chaque paquet

Paquet	Responsabilité	Ne fait jamais
core	navigation, session, thème, exécution asynchrone, libellés	parler à l'API, afficher une vue
api	appels HTTP/JSON, erreurs normalisées, DTO immuables	décider d'un droit, formater de l'interface
api·mock	jeu de réponses simulées pour maquetter sans serveur	être livré en production
model	modèles observables (propriétés JavaFX) prêts à lier à la vue	contenir une règle métier
ui·commun	barre de navigation, badges, cartes, dialogues, états vides	connaître un écran particulier
ui·apprenant	8 écrans du parcours : inscription → résultat	accéder aux écrans enseignant/admin
ui·enseignant	éditeur de cours, éditeur de test, conditions, résultats	modifier une note déjà produite
ui·admin	comptes, journal d'audit, agent IA	voir le contenu pédagogique
ui·ia	panneau d'assistance, masquable par CSS	écrire une note ou valider un niveau

2.2 Le trajet d'un clic (exemple : « exécuter »)

```
ExerciceController.onExecuter()
  → TaskRunner.run( () -> TestApi.executer(code, langage) )
      → ApiClient POST /api/tentatives/execution
          → serveur : conteneur isolé (5 s / 512 Mo)
      ← ResultatExecution(sortie, erreurs, statut)
  ← Platform.runLater( console.afficher( ... ) )
  ← CSS : .console .erreur  (ligne fautive surlignée)
```

La vue ne calcule rien : elle affiche l'état renvoyé. C'est la traduction du principe « **la décision est côté serveur** » (BNF-T6).

> **Règle de revue de code n°1 :** si un fichier `ui/*Controller.java` contient un `if` sur un **droit** ou sur un **seuil**, la règle est au mauvais endroit — elle doit descendre dans le service serveur.

3 · Correspondance écran ↔ fichiers ↔ style

Ce tableau relie les huit modèles de tâches (HTA) aux fichiers réels : c'est la table de correspondance à utiliser pour découper le travail entre développeurs.

Modèle	Écran	Fichiers (dans `ui/…`)	Style / ressource	Contrôle dominant
A1	Inscription + choix du parcours	apprenant/inscription/* · commun/SelecteurParcours.java	.inscription · .card-parcours · badge-langage	TabPane · TextField
A2	Cours + exercice	apprenant/cours/* · apprenant/exercice/* (EditeurCode, ConsoleSortie)	syntax-java / -python / -cpp · .console	TreeView · TextArea
A3	Test de validation	apprenant/test/* (TestController, Minuteur)	.test-mode (assistant masqué) · .chrono	BorderPane · ListView
A4	Tableau de bord apprenant	apprenant/tableaubord/* · apprenant/parcours/FriseNiveaux.java	.frise · .etat-valide / -en-cours / -verrouille	TabPane · Pane
E1	Éditeur de niveau	enseignant/editeurcours/* (Autosave) · editeurtest/* · conditions/*	.editor · .conditions · .autosave	SplitPane · TableView

E2	Résultats d'un niveau	enseignant/resultats/*	.kpi · .etat-alerte · .diff	TableView · cartes KPI
AD1	Comptes et droits	admin/ComptesView · admin/JournalAuditView · admin/AgentIaView	.admin · .action-danger · .audit	TableView · Dialog
IA1	Panneau d'assistance	ia/PanneauAssistant.fxml + controller	.panel-ia (masquable par classe)	VBox défilante

Ce que cette table garantit : chaque tâche modélisée a **un dossier, un FXML, un contrôleur et un bloc CSS**. Une tâche sans dossier est une tâche non implémentée ; un dossier sans tâche est du code inutile.

4 · Les ressources : style, thèmes, textes, images

4.1 Les trois feuilles CSS et leur ordre d'application

```
// App.java — l'ordre compte
scene.getStylesheets().addAll(
    ThemeManager.themeCss(Session.prefTheme()),   // 1. le thème AtlantaFX (clair ou sombre)
    css("app.css"),                               // 2. nos jetons et nos composants
    ThemeManager.surchargeCss(Session.prefTheme())// 3. la surcharge propre au thème
);
```

Fichier	Contenu	Qui le modifie
AtlantaFX (theme-light / theme-dark)	fourni par la bibliothèque : couleurs de base de tous les contrôles	personne — mis à jour avec la bibliothèque
css/app.css	jetons de design (`-color-java` , `-radius-card` , `-space-2`) et classes des composants maison	le référent IHM (seul propriétaire)
css/theme-light.css · theme-dark.css	nos surcharges : contraste des badges, lisibilité du code, états des niveaux	le référent IHM — les deux fichiers restent alignés
css/syntax/syntax-*.css	coloration par langage : `.kw` , `.str` , `.num` , `.com`	un fichier par langage, jamais mélangés (IHM-22)

4.2 Textes, images et états

Ressource	Règle de rangement	Conséquence si on l'oublie
i18n/messages_fr.properties	une clé par libellé, préfixée par l'écran : `resultat.seuil=Seuil`	le même mot écrit de trois façons (IHM-11)
img/etats/	un pictogramme par état : `valide.svg` , `en-cours.svg` , `verrouille.svg`	état porté par la couleur seule (IHM-15)
img/icones/	icônes d'action en SVG monochrome, colorées par CSS	icônes illisibles en thème sombre
css/syntax/	un fichier par langage, mêmes noms de classes	code C++ coloré comme du Python (IHM-22)

5 · Ordre de création : quoi écrire en premier

L'arborescence complète fait peur ; elle se construit dans un ordre qui laisse toujours l'application **exécutable**. Chaque étape ci-dessous se termine par une démonstration possible.

Étape	Fichiers à créer	Ce que l'on obtient	Démonstration
1	pom.xml · module-info.java · App.java · app.css · ThemeManager	une fenêtre vide, thème AtlantaFX clair/sombre, jetons en place	bascule de thème en direct
2	core/Router · core/Session · ui/commun/NavBar	une navigation entre 3 écrans vides, fil d'Ariane	cliquer dans les 3 rôles
3	api/ApiClient · api/mock · model/*	des écrans alimentés par des données simulées	frise des niveaux remplie
4	apprenant/cours · exercice (EditeurCode, ConsoleSortie)	le cœur du produit : lire, coder, exécuter	erreur de compilation affichée ligne par ligne
5	apprenant/test · resultat	la boucle de validation complète	note + seuil + points manquants
6	enseignant/editeurcours · conditions	le contenu et les règles deviennent paramétrables	changer un seuil sans redéploiement
7	admin/* · ia/PanneauAssistant	les droits et l'aide optionnelle	désactiver l'assistant, journal d'audit
8	src/test/java · tools/	tests TestFX + installateur	application lancée sur un poste vierge

5.1 Conventions de nommage

Élément	Convention
Écran	`NomView.fxml` + `NomController.java` (même nom)
Composant maison	substantif en français : `FriseNiveaux`, `BadgeLangage`
Classe CSS	minuscules, tirets, une classe par intention : `.etat-verrouille`
Paquet	un paquet = un rôle ou un module fonctionnel, jamais un type d'objet
DTO	suffixe `Dto`, immuable (`record`), jamais exposé à la vue directement
Modèle de vue	suffixe `VM`, propriétés JavaFX pour la liaison

5.2 Checklist de revue (5 lignes)

① aucun `setStyle()` non justifié ;
② aucun libellé en dur dans le FXML ou le contrôleur ;
③ aucun appel réseau sur le FX Application Thread ;
④ aucun contrôle de droit ou de seuil dans `ui/` ;
⑤ chaque écran a son bloc CSS et ses deux captures (clair + sombre).

Cette liste reprend les cinq contraintes les plus souvent violées dans un projet JavaFX ; elle tient en une revue de 5 minutes par écran.